

Informe Laboratorio 5

Sección 1

Alumno Felipe Maturana
e-mail: felipe.maturana1@mail.udp.cl

Junio 2026

Índice

Descripción de actividades	4
1. Desarrollo (Parte 1)	6
1.1. Códigos de cada Dockerfile	6
1.1.1. C1	6
1.1.2. C2	7
1.1.3. C3	7
1.1.4. C4/S1	8
1.1.5. Docker Compose	8
1.1.6. Comandos para levantar el entorno	10
1.2. Creación de las credenciales para S1	10
1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	10
1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	11
1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	12
1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	13
1.7. Compara la versión de HASSH obtenida con la base de datos para validar si el cliente corresponde al mismo	13
1.8. Tipo de información contenida en cada uno de los paquetes generados en texto plano	13
1.8.1. C1	14
1.8.2. C2	14
1.8.3. C3	14
1.8.4. C4/S1	14
1.9. Diferencia entre C1 y C2	14
1.10. Diferencia entre C2 y C3	15
1.11. Diferencia entre C3 y C4	15
2. Desarrollo (Parte 2)	16
2.1. Identificación del cliente SSH con versión “?”	16
2.2. Replicación de tráfico al servidor (paso por paso)	16
3. Desarrollo (Parte 3)	18
3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)	18
4. Desarrollo (Parte 4)	20
4.1. Explicación OpenSSH en general	20
4.2. Capas de Seguridad en OpenSSH	21

4.3. Identificación de que protocolos no se cumplen	21
---	----

Descripción de actividades

Para este último laboratorio, se solicita trabajar con Docker y el protocolo SSH, a fin de poder entender el concepto de criptografía asimétrica y firmas digitales.

Para lo anterior deberá:

- Crear 4 contenedores en Docker por medio de un DockerFile, donde cada uno tendrá el siguiente SO: Ubuntu 16.10, Ubuntu 18.10, Ubuntu 20.10 y Ubuntu 22.10 a los cuales se llamarán C1, C2, C3 y C4 respectivamente.
El equipo con Ubuntu 22.10 también será utilizado como S1.
- Para cada uno de ellos, deberá instalar el cliente openSSH disponible en los repositorios de apt, y para el equipo S1 deberá también instalar el servidor openSSH.
- En S1 deberá crear el usuario “**prueba**” con contraseña “**prueba**”, para acceder a él desde los clientes por el protocolo SSH.
- En total serán 4 escenarios, donde cada uno corresponderá a los siguientes equipos:
 - C1 → S1
 - C2 → S1
 - C3 → S1
 - C4 → S1

Pasos:

1. Para cada uno de los 4 escenarios, deberá capturar el tráfico generado por cada conexión con el server. A partir de cada handshake, deberá analizar el patrón de tráfico generado por cada cliente y adicionalmente obtener el HASSH que lo identifique. De esta forma podrá obtener una huella digital para cada cliente a partir de su tráfico. Cada HASSH deberá compararlo con la base de datos HASSH disponible en el módulo de TLS, e identificar si el hash obtenido corresponde a la misma versión de su cliente.

Indique el tamaño de los paquetes del flujo generados por el cliente y el contenido asociado a cada uno de ellos. Indique qué información distinta contiene el escenario siguiente (diff incremental). El objetivo de este paso es identificar claramente los cambios entre las distintas versiones de ssh.

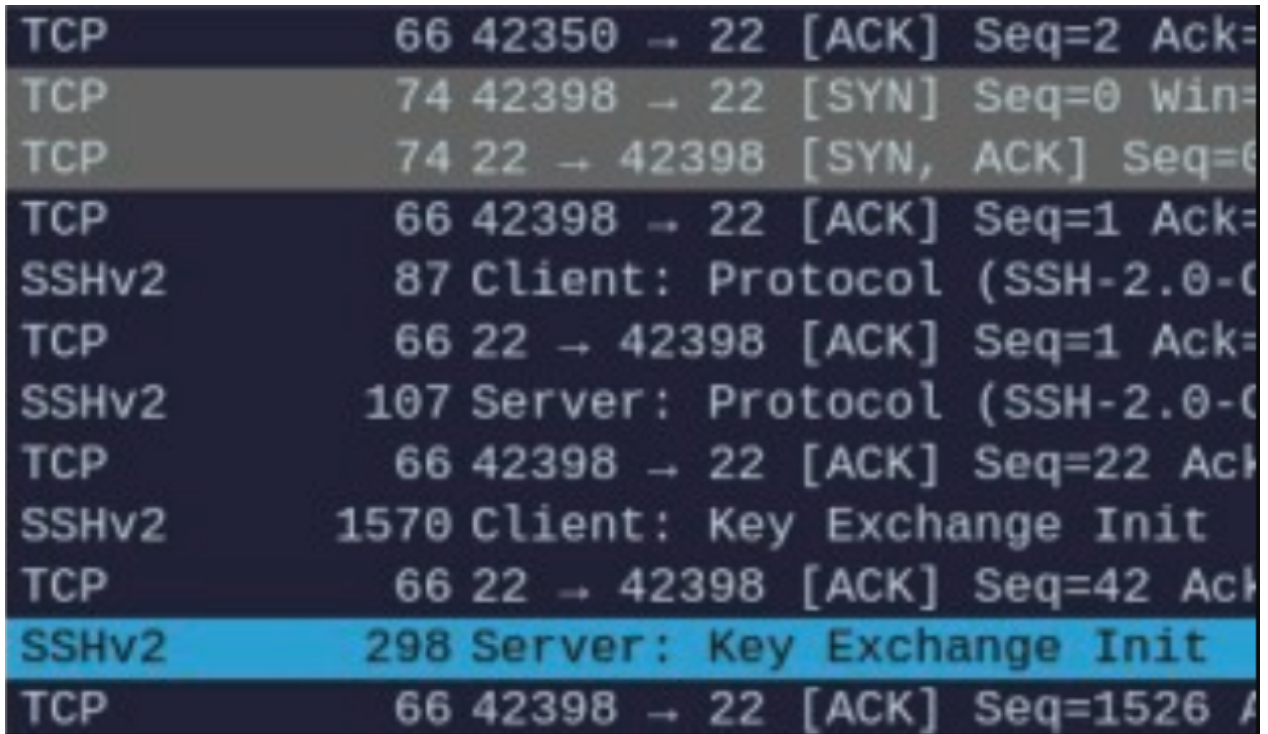
2. Para poder identificar que el usuario efectivamente es el informante, éste utilizará una versión única de cliente. ¿Con qué cliente SSH se habrá generado el siguiente tráfico?

Protocol	Length	Info
TCP	74	34328 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=14
TCP	66	34328 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0
SSHv2	85	Client: Protocol (SSH-2.0-OpenSSH_?)
TCP	66	34328 → 22 [ACK] Seq=20 Ack=42 Win=64256 Len=
SSHv2	1578	Client: Key Exchange Init
TCP	66	34328 → 22 [ACK] Seq=1532 Ack=1122 Win=64128
SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exc
TCP	66	34328 → 22 [ACK] Seq=1580 Ack=1574 Win=64128
SSHv2	82	Client: New Keys
SSHv2	110	Client: Encrypted packet (len=44)
TCP	66	34328 → 22 [ACK] Seq=1640 Ack=1618 Win=64128
SSHv2	126	Client: Encrypted packet (len=60)
TCP	66	34328 → 22 [ACK] Seq=1700 Ack=1670 Win=64128
SSHv2	150	Client: Encrypted packet (len=84)
TCP	66	34328 → 22 [ACK] Seq=1784 Ack=1698 Win=64128
SSHv2	178	Client: Encrypted packet (len=112)
TCP	66	34328 → 22 [ACK] Seq=1896 Ack=2198 Win=64128

Figura 1: Tráfico generado del informante

Replique este tráfico generado en la imagen. Debe generar el tráfico con la misma versión resaltada en azul. Recuerde que toda la información generada es parte del sw, por lo tanto usted puede modificar toda la información.

3. Para que el informante esté seguro de nuestra identidad, nos pide que el patrón del tráfico de nuestro server también sea modificado, hasta que el Key Exchange Init del server sea menor a 300 bytes. Indique qué pasos realizó para lograr esto.



TCP	66	42350 → 22	[ACK] Seq=2 Ack=
TCP	74	42398 → 22	[SYN] Seq=0 Win=
TCP	74	22 → 42398	[SYN, ACK] Seq=0
TCP	66	42398 → 22	[ACK] Seq=1 Ack=
SSHv2	87	Client: Protocol (SSH-2.0-C	
TCP	66	22 → 42398	[ACK] Seq=1 Ack=
SSHv2	107	Server: Protocol (SSH-2.0-C	
TCP	66	42398 → 22	[ACK] Seq=22 Ack=
SSHv2	1570	Client: Key Exchange Init	
TCP	66	22 → 42398	[ACK] Seq=42 Ack=
SSHv2	298	Server: Key Exchange Init	
TCP	66	42398 → 22	[ACK] Seq=1526 A

Figura 2: Captura del Key Exchange

- Tomando en cuenta lo aprendido en este laboratorio, así como en los anteriores, explique el protocolo OpenSSH y las diferentes capas de seguridad que son parte del protocolo para garantizar los principios de seguridad de la información, integridad, confidencialidad, disponibilidad, autenticidad y no repudio. Es importante que sea muy específico en el objetivo del principio en el protocolo. En caso de considerar que alguno de los principios no se cumple, justifique su razonamiento. Es fundamental que su análisis se base en el tráfico SSH interceptado.

1. Desarrollo (Parte 1)

1.1. Códigos de cada Dockerfile

Para realizar esta experiencia, se crearon cuatro Dockerfiles que instalan el cliente de OpenSSH en diferentes versiones de Ubuntu, además de configurar el servidor en la versión más reciente (C4/S1).

1.1.1. C1

```
FROM ubuntu:16.10
```

```
# Adjust sources for EOL version
RUN sed -i -e 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.
    list && \
    sed -i -e 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources
    .list

RUN apt-get update && apt-get install -y \
    openssh-client \
    iputils-ping \
    tcpdump \
    && rm -rf /var/lib/apt/lists/*

CMD ["bash"]
```

1.1.2. C2

```
FROM ubuntu:18.10

# Adjust sources for EOL version
RUN sed -i -e 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.
    list && \
    sed -i -e 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources
    .list

RUN apt-get update && apt-get install -y \
    openssh-client \
    iputils-ping \
    tcpdump \
    && rm -rf /var/lib/apt/lists/*

CMD ["bash"]
```

1.1.3. C3

```
FROM ubuntu:20.10

# Adjust sources for EOL version
RUN sed -i -e 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.
    list && \
    sed -i -e 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources
    .list

RUN apt-get update && apt-get install -y \
    openssh-client \
```

```
iputils-ping \  
tcpdump \  
&& rm -rf /var/lib/apt/lists/*  
  
CMD ["bash"]
```

1.1.4. C4/S1

```
FROM ubuntu:22.10  
  
# Adjust sources for EOL version  
RUN sed -i -e 's/archive.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources.  
list && \  
sed -i -e 's/security.ubuntu.com/old-releases.ubuntu.com/g' /etc/apt/sources  
.list  
  
RUN apt-get update && apt-get install -y \  
openssh-server \  
openssh-client \  
iputils-ping \  
tcpdump \  
&& rm -rf /var/lib/apt/lists/*  
  
# Create user 'prueba' with password 'prueba'  
RUN useradd -m -s /bin/bash prueba && echo "prueba:prueba" | chpasswd  
  
RUN mkdir -p /var/run/sshd  
  
EXPOSE 22  
  
CMD ["/usr/sbin/sshd", "-D"]
```

1.1.5. Docker Compose

```
services:  
  s1:  
    build:  
      context: ./C4  
      network: host  
    image: c4  
    container_name: s1  
    networks:  
      ssh_net:  
        ipv4_address: 172.21.0.10
```



```
c1:
  build:
    context: ./C1
    network: host
  image: c1
  container_name: c1
  tty: true
  stdin_open: true
  depends_on:
    - s1
  networks:
    ssh_net:
      ipv4_address: 172.21.0.11

c2:
  build:
    context: ./C2
    network: host
  image: c2
  container_name: c2
  tty: true
  stdin_open: true
  depends_on:
    - s1
  networks:
    ssh_net:
      ipv4_address: 172.21.0.12

c3:
  build:
    context: ./C3
    network: host
  image: c3
  container_name: c3
  tty: true
  stdin_open: true
  depends_on:
    - s1
  networks:
    ssh_net:
      ipv4_address: 172.21.0.13

c4:
  build:
    context: ./C4
```

```
    network: host
    image: c4
    container_name: c4_client
    depends_on:
      - s1
    networks:
      ssh_net:
        ipv4_address: 172.21.0.14

networks:
  ssh_net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.21.0.0/24
```

1.1.6. Comandos para levantar el entorno

Para construir y levantar los contenedores se utilizó el siguiente comando desde el directorio `src`:

```
docker compose up -d --build
```

1.2. Creación de las credenciales para S1

Para el servidor S1, la creación del usuario se configuró de manera automática directamente en su Dockerfile. Se utilizó el siguiente comando para crear el usuario **prueba** con la contraseña **prueba** y asignarles su directorio personal:

```
RUN useradd -m -s /bin/bash prueba && echo "prueba:prueba" | chpasswd
```

1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Para el cliente C1 (Ubuntu 16.10), se capturó el flujo de tráfico hacia el servidor S1. El proceso de handshake se divide en las siguientes etapas principales:

- **Protocol Exchange (Rojo)**: Intercambio de versiones de software entre cliente y servidor.
- **Key Exchange Init (Verde)**: Negociación de algoritmos de cifrado, compresión y hashing.
- **Elliptic Curve Diffie-Hellman (Naranja)**: Proceso de generación y compartición de llaves para el canal seguro.

1.4 Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

No.	Time	Source	Destination	Protocol	Length	Info
4	0.000535940	172.21.0.11	172.21.0.10	SSHv2	107	Client: Protocol (SSH-2.0-OpenSSH_7.3p1 Ubuntu-1ubuntu0.1)
6	0.001595575	172.21.0.10	172.21.0.11	SSHv2	107	Server: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-1ubuntu7.3)
8	0.002882038	172.21.0.11	172.21.0.10	SSHv2	1498	Client: Key Exchange Init
9	0.003885454	172.21.0.10	172.21.0.11	SSHv2	1146	Server: Key Exchange Init
10	0.006696439	172.21.0.11	172.21.0.10	SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
11	0.013748945	172.21.0.10	172.21.0.11	SSHv2	662	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
12	0.017650168	172.21.0.11	172.21.0.10	SSHv2	82	Client: New Keys
14	0.057798683	172.21.0.11	172.21.0.10	SSHv2	110	Client:
16	0.057959922	172.21.0.10	172.21.0.11	SSHv2	110	Server:
17	0.058199396	172.21.0.11	172.21.0.10	SSHv2	134	Client:
18	0.06684181	172.21.0.10	172.21.0.11	SSHv2	118	Server:
24	53.936778887	172.21.0.11	172.21.0.10	SSHv2	214	Client:

Figura 3: Handshake detallado y captura de paquetes para C1

El primer paquete enviado por el cliente para la negociación (KEXINIT) tiene un tamaño de **1498 bytes**. A partir de los algoritmos propuestos por este cliente, se obtuvo el siguiente HASSH:

- **HASSH:** 0e4584cb9f2dd077dbf8ba0df8112d8e

```

- Key Exchange (method:curve25519-sha256@libssh.org)
  Message Code: Key Exchange Init (20)
  Algorithms
    Cookie: d0591a01c51819e889c02db518ad9202
    kex_algorithms length: 286
    kex_algorithms string [truncated]: curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521
    server_host_key_algorithms length: 290
    server_host_key_algorithms string [truncated]: ecdsa-sha2-nistp256-cert-v01@openssh.com,ecdsa-sha2-nistp256,ecdsa-sha2-nistp384,ecdsa-sha2-nistp521
    encryption_algorithms_client_to_server length: 150
    encryption_algorithms_client_to_server string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-gcm@openssh.com
    encryption_algorithms_server_to_client length: 150
    encryption_algorithms_server_to_client string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256-gcm@openssh.com
    mac_algorithms_client_to_server length: 213
    mac_algorithms_client_to_server string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha1,hmac-sha2-256,hmac-sha2-512
    mac_algorithms_server_to_client length: 213
    mac_algorithms_server_to_client string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac-sha1,hmac-sha2-256,hmac-sha2-512
    compression_algorithms_client_to_server length: 26
    compression_algorithms_client_to_server string: none,zlib@openssh.com,zlib
    compression_algorithms_server_to_client length: 26
    compression_algorithms_server_to_client string: none,zlib@openssh.com,zlib
    languages_client_to_server length: 0
    languages_client_to_server string:
    languages_server_to_client length: 0
    languages_server_to_client string:
    First KEX Packet Follows: 0
    Reserved: 00000000
    [hashAlgorithms [truncated]: curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521]
    [hashsh: 0e4584cb9f2dd077dbf8ba0df8112d8e]
    Padding String: 000000000000000000000000
    Sequence number: 0

```

Figura 4: Detalle de algoritmos y HASSH para C1

1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Para el cliente C2 (Ubuntu 18.10), el proceso de handshake sigue la misma estructura lógica de intercambio de protocolos y negociación de llaves. En este escenario, el paquete KEXINIT presenta un tamaño de **1426 bytes**.

- **HASSH:** 06046964c022c6407d15a27b12a6a4fb

No.	Time	Source	Destination	Protocol	Length	Info
4	0.000337772	172.21.0.12	172.21.0.10	SSHv2	178	Client: Protocol (SSH-2.0-OpenSSH_7.7p1 Ubuntu-1ubuntu0.3)
6	0.001679990	172.21.0.10	172.21.0.12	SSHv2	107	Server: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-1ubuntu7.3)
8	0.001987493	172.21.0.12	172.21.0.10	SSHv2	1426	Client: Key Exchange Init
9	0.003971172	172.21.0.10	172.21.0.12	SSHv2	1146	Server: Key Exchange Init
10	0.006798682	172.21.0.12	172.21.0.10	SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
11	0.013776835	172.21.0.10	172.21.0.12	SSHv2	662	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
13	4.553694100	172.21.0.12	172.21.0.10	SSHv2	82	Client: New Keys
15	4.593974267	172.21.0.12	172.21.0.10	SSHv2	110	Client:

Figura 5: Captura de paquetes para C2

1.5 Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

```

~ Algorithms
Cookie: c79bf3c7c5ce3aec735cb28c6efa5bec
kex_algorithms length: 304
kex_algorithms string [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh
server_host_key_algorithms length: 290
server_host_key_algorithms string [truncated]: ecdsa-sha2-nistp256-cert-v01@openssh.com,ecdsa-sha2-nistp256
encryption_algorithms_client_to_server length: 108
encryption_algorithms_client_to_server string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256
encryption_algorithms_server_to_client length: 108
encryption_algorithms_server_to_client string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256
mac_algorithms_client_to_server length: 213
mac_algorithms_client_to_server string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac
mac_algorithms_server_to_client length: 213
mac_algorithms_server_to_client string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac
compression_algorithms_client_to_server length: 26
compression_algorithms_client_to_server string: none,zlib@openssh.com,zlib
compression_algorithms_server_to_client length: 26
compression_algorithms_server_to_client string: none,zlib@openssh.com,zlib
languages_client_to_server length: 0
languages_client_to_server string:
languages_server_to_client length: 0
languages_server_to_client string:
First KEX Packet Follows: 0
Reserved: 00000000
[hashshAlgorithms [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2
[hashsh: 66846964c022c6407d15a27b12a6a4fb]
Padding String: 00000000000000000000000000000000

```

Figura 6: Detalle de algoritmos y HASSH para C2

1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

El cliente C3 (Ubuntu 20.10) muestra un incremento en la complejidad de los algoritmos soportados. El tamaño del paquete KEXINIT es de **1578 bytes**, siendo el más extenso de los analizados hasta ahora debido a la inclusión de nuevos métodos de intercambio.

- **HASSH: ae8bd7dd09970555aa4c6ed22adbbf56**

No.	Time	Source	Destination	Protocol	Length	Info
0	0.00144802	172.21.0.10	172.21.0.10	SSH2	107	Server: Protocol (SSH-2.0-OpenSSH_8.3p1 Ubuntu-1ubuntu0.1)
8	0.013568408	172.21.0.13	172.21.0.10	SSH2	157	Client: Protocol (SSH-2.0-OpenSSH_8.3p1 Ubuntu-1ubuntu0.1)
10	0.014128619	172.21.0.13	172.21.0.10	SSH2	1578	Client: Key Exchange Init
12	0.016214648	172.21.0.10	172.21.0.13	SSH2	1146	Server: Key Exchange Init
13	0.019171705	172.21.0.13	172.21.0.10	SSH2	114	Client: Elliptic Curve Diffie-Hellman Key Exchange Init
14	0.026318995	172.21.0.10	172.21.0.13	SSH2	662	Server: Elliptic Curve Diffie-Hellman Key Exchange Reply, New Keys
16	2.293961637	172.21.0.13	172.21.0.10	SSH2	62	Client: New Keys
18	2.334064307	172.21.0.13	172.21.0.10	SSH2	118	Client:

Figura 7: Captura de paquetes para C3

```

~ Algorithms
Cookie: 45a80dc0494517fea022c52b12a0af36
kex_algorithms length: 241
kex_algorithms string [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh
server_host_key_algorithms length: 500
server_host_key_algorithms string [truncated]: ecdsa-sha2-nistp256-cert-v01@openssh.com,ecdsa-sha2-nistp256
encryption_algorithms_client_to_server length: 108
encryption_algorithms_client_to_server string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256
encryption_algorithms_server_to_client length: 108
encryption_algorithms_server_to_client string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256
mac_algorithms_client_to_server length: 213
mac_algorithms_client_to_server string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac
mac_algorithms_server_to_client length: 213
mac_algorithms_server_to_client string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac
compression_algorithms_client_to_server length: 26
compression_algorithms_client_to_server string: none,zlib@openssh.com,zlib
compression_algorithms_server_to_client length: 26
compression_algorithms_server_to_client string: none,zlib@openssh.com,zlib
languages_client_to_server length: 0
languages_client_to_server string:
languages_server_to_client length: 0
languages_server_to_client string:
First KEX Packet Follows: 0
Reserved: 00000000
[hashshAlgorithms [truncated]: curve25519-sha256,curve25519-sha256@libssh.org,ecdh-sha2-nistp256,ecdh-sha2
[hashsh: ae8bd7dd09970555aa4c6ed22adbbf56]
Padding String: 00000000000000000000000000000000

```

Figura 8: Detalle de algoritmos y HASSH para C3

1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Finalmente, para C4 (Ubuntu 22.10), el paquete KEXINIT tiene un tamaño de **1570 by-tes**. Se observa la presencia de algoritmos modernos como `sntrup761x25519-sha512@openssh.com`.

- **HASSH: 78c05d999799066a2b4554ce7b1585a6**

No.	Time	Source	Destination	Protocol	Length	Info
0	0.00049911	172.21.0.14	172.21.0.10	SSHv2	107	Client: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-ubuntu/2.3)
0	0.00111609	172.21.0.10	172.21.0.14	SSHv2	1578	Server: Protocol (SSH-2.0-OpenSSH_9.0p1 Ubuntu-ubuntu/2.3)
10	0.001920578	172.21.0.14	172.21.0.10	SSHv2	1578	Client: Key Exchange Init
12	0.003396266	172.21.0.10	172.21.0.14	SSHv2	1146	Server: Key Exchange Init
14	0.123807935	172.21.0.14	172.21.0.10	SSHv2	1274	Client: Diffie-Hellman Key Exchange Init
15	0.147938457	172.21.0.10	172.21.0.14	SSHv2	1638	Server: Diffie-Hellman Key Exchange Reply, New Keys
17	5.908594979	172.21.0.14	172.21.0.10	SSHv2	82	Client: New Keys
19	5.949231445	172.21.0.14	172.21.0.10	SSHv2	118	Client: New Keys
21	5.949359563	172.21.0.10	172.21.0.14	SSHv2	118	Server: New Keys
22	5.949464662	172.21.0.14	172.21.0.10	CCPv0	154	Client: New Keys

Figura 9: Captura de paquetes para C4

```

Algorithms
Cookie: 78cbf05e2f07ceab2c3336001b0656e5
kex_algorithms length: 276
kex_algorithms string [truncated]: sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256
server_host_key_algorithms length: 463
server_host_key_algorithms string [truncated]: ssh-ed25519-cert-v01@openssh.com,ecdsa-sha2-nistp256-cert-
encryption_algorithms_client_to_server length: 108
encryption_algorithms_client_to_server string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256
encryption_algorithms_server_to_client length: 108
encryption_algorithms_server_to_client string: chacha20-poly1305@openssh.com,aes128-ctr,aes192-ctr,aes256
mac_algorithms_client_to_server length: 213
mac_algorithms_client_to_server string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac
mac_algorithms_server_to_client length: 213
mac_algorithms_server_to_client string [truncated]: umac-64-etm@openssh.com,umac-128-etm@openssh.com,hmac
compression_algorithms_client_to_server length: 26
compression_algorithms_client_to_server string: none,zlib@openssh.com,zlib
compression_algorithms_server_to_client length: 26
compression_algorithms_server_to_client string: none,zlib@openssh.com,zlib
languages_client_to_server length: 0
languages_client_to_server string:
languages_server_to_client length: 0
languages_server_to_client string:
First KEX Packet Follows: 0
Reserved: 00000000
[hashsh: 78c05d999799066a2b4554ce7b1585a6]
Padding String: 00000000
Sequence number: 0

```

Figura 10: Detalle de algoritmos y HASSH para C4

1.7. Compara la versión de HASSH obtenida con la base de datos para validar si el cliente corresponde al mismo

1.8. Tipo de información contenida en cada uno de los paquetes generados en texto plano

Durante la fase inicial del protocolo SSH (Handshake), existe información que viaja sin cifrar para permitir la negociación entre los extremos. A continuación se detalla la información legible para cada escenario.

1.8.1. C1

En el caso de C1, la información en texto plano expone la versión del software `OpenSSH_7.3p1` y el sistema operativo `Ubuntu-1ubuntu0.1`. Además, en el paquete *KEXINIT*, es posible leer la lista completa de algoritmos de cifrado simétrico (como `aes128-ctr` y `chacha20-poly1305`), algoritmos de intercambio de llaves y métodos de compresión soportados por el cliente.

1.8.2. C2

Para C2, el banner inicial identifica al cliente como `OpenSSH_7.7p1`. Al igual que en el caso anterior, los paquetes de negociación inicial muestran en texto claro las capacidades del sistema, permitiendo identificar que esta versión ha comenzado a omitir ciertos algoritmos antiguos de 64 bits en comparación a C1, aunque la estructura del mensaje sigue siendo totalmente legible antes del envío del paquete *New Keys*.

1.8.3. C3

El cliente C3 se identifica en texto plano como `OpenSSH_8.3p1`. En sus paquetes iniciales se observa la inclusión de nuevos algoritmos de intercambio de llaves con nombres extensos, los cuales son perfectamente visibles mediante un análisis de tráfico simple. Esta información permite a un observador externo conocer con precisión la suite criptográfica que el cliente está dispuesto a utilizar.

1.8.4. C4/S1

En este escenario, el cliente se identifica como `OpenSSH_9.0p1`. Al actuar también como servidor (S1), este equipo envía adicionalmente su propia identificación de versión y su *Host Key* pública durante el intercambio inicial. Toda esta metadata, incluyendo el soporte para algoritmos post-cuánticos como `sntrup761x25519`, viaja en formato legible hasta que se confirma la generación de las llaves compartidas.

1.9. Diferencia entre C1 y C2

Al comparar el tráfico de C1 (OpenSSH 7.3) con C2 (OpenSSH 7.7), se observa una **disminución en el tamaño del paquete KEXINIT**, pasando de 1498 bytes a 1426 bytes.

Técnicamente, este "diff" se explica por la limpieza de algoritmos obsoletos. En C2, se eliminaron de la lista de negociación por defecto todos los cifrados de bloque en modo CBC (`aes128-cbc`, `aes256-cbc`) y el antiguo `3des-cbc`. Aunque se añadieron variantes de curvas elípticas, la eliminación de estos hilos de texto largos redujo el tamaño total del paquete y cambió drásticamente el HASH de `0e45...` a `0604...`.

1.10. Diferencia entre C2 y C3

La transición de C2 (OpenSSH 7.7) a C3 (OpenSSH 8.3) marca un **incremento significativo en el tamaño del paquete**, subiendo a 1578 bytes (una diferencia de +152 bytes).

El cambio principal radica en la expansión de la lista `server_host_key_algorithms`, la cual pasó de 290 a 500 bytes de longitud. Esto se debe a la introducción de soporte para llaves de seguridad físicas (FIDO/U2F), añadiendo algoritmos como `sk-ecdsa-sha2-nistp256-cert-v01@open`. Esta expansión de capacidades de autenticación es lo que define la huella digital (HASSH) de la versión 8.3.

1.11. Diferencia entre C3 y C4

Finalmente, entre C3 (OpenSSH 8.3) y C4 (OpenSSH 9.0), el paquete experimenta una leve reducción de tamaño a 1570 bytes.

La diferencia más notable en este escenario es la introducción del algoritmo **post-cuántico** `sntrup761x25519-sha512@openssh.com` como la opción de mayor prioridad en los `kex_algorithms`. A pesar de añadir este nombre tan extenso, el tamaño total bajó ligeramente debido a una nueva depuración en los algoritmos de llaves de host, eliminando variantes menos utilizadas o redundantes de certificados. Este cambio de prioridades y la inclusión del método post-cuántico genera el HASSH final de `78c0...`

2. Desarrollo (Parte 2)

2.1. Identificación del cliente SSH con versión “?”

A partir del análisis de la captura de tráfico proporcionada en la Figura 1, se puede determinar la versión del cliente SSH utilizado por el informante mediante la observación del patrón de tráfico, específicamente el tamaño del paquete *Key Exchange Init*.

En la captura se observa que el paquete **Client: Key Exchange Init** tiene un tamaño exacto de **1578 bytes**. Al contrastar este valor con los resultados obtenidos en la Parte 1 de este laboratorio, se evidencia que este tamaño coincide plenamente con el generado por el contenedor **C3**, el cual utiliza la versión **OpenSSH 8.3p1** sobre Ubuntu 20.10. Por lo tanto, se concluye que el informante utilizó esta versión específica para realizar la conexión.

2.2. Replicación de tráfico al servidor (paso por paso)

Para replicar el tráfico del informante con la versión **OpenSSH_?**, se descargó, modificó y compiló OpenSSH desde el código fuente dentro del contenedor C3. A continuación se detalla cada paso realizado.

Paso 1: Descarga del código fuente. Dentro del contenedor, se navegó al directorio `/home/openssh/` y se descargó el tarball de OpenSSH 9.6p1 directamente desde el servidor oficial de OpenBSD.

```
root@aefc862e4624:/# cd home
root@aefc862e4624:/home# cd openssh/
root@aefc862e4624:/home/openssh# ls
root@aefc862e4624:/home/openssh# wget https://cdn.openbsd.org/pub/OpenBSD/OpenSSH/portable/openssh-9.6p1.tar.gz
```

Figura 11: Descarga del código fuente de OpenSSH 9.6p1

Paso 2: Descompresión y acceso al directorio. Se descomprimió el archivo descargado con `tar xzf` y se ingresó al directorio resultante `openssh-9.6p1/`.

```
root@aefc862e4624:/home/openssh# ls
openssh-9.6p1.tar.gz
root@aefc862e4624:/home/openssh# tar xzf openssh-9.6p1.tar.gz
root@aefc862e4624:/home/openssh# cd openssh-9.6p1/
```

Figura 12: Descompresión del tarball y acceso al directorio fuente

Paso 3: Apertura del archivo `version.h` con `vim`. Se abrió el archivo `version.h` con el editor `vim` para localizar la línea que define la cadena de versión del cliente SSH.

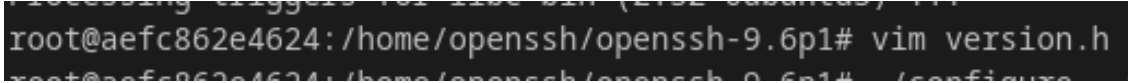


Figura 13: Apertura de version.h con vim

Paso 4: Modificación de la versión en version.h. Se editó la macro SSH_VERSION cambiando su valor de `."openSSH.9.6"` a `."openSSH.?"`, tal como se observa en la figura. Esta es la cadena que el cliente enviará durante el handshake SSH.

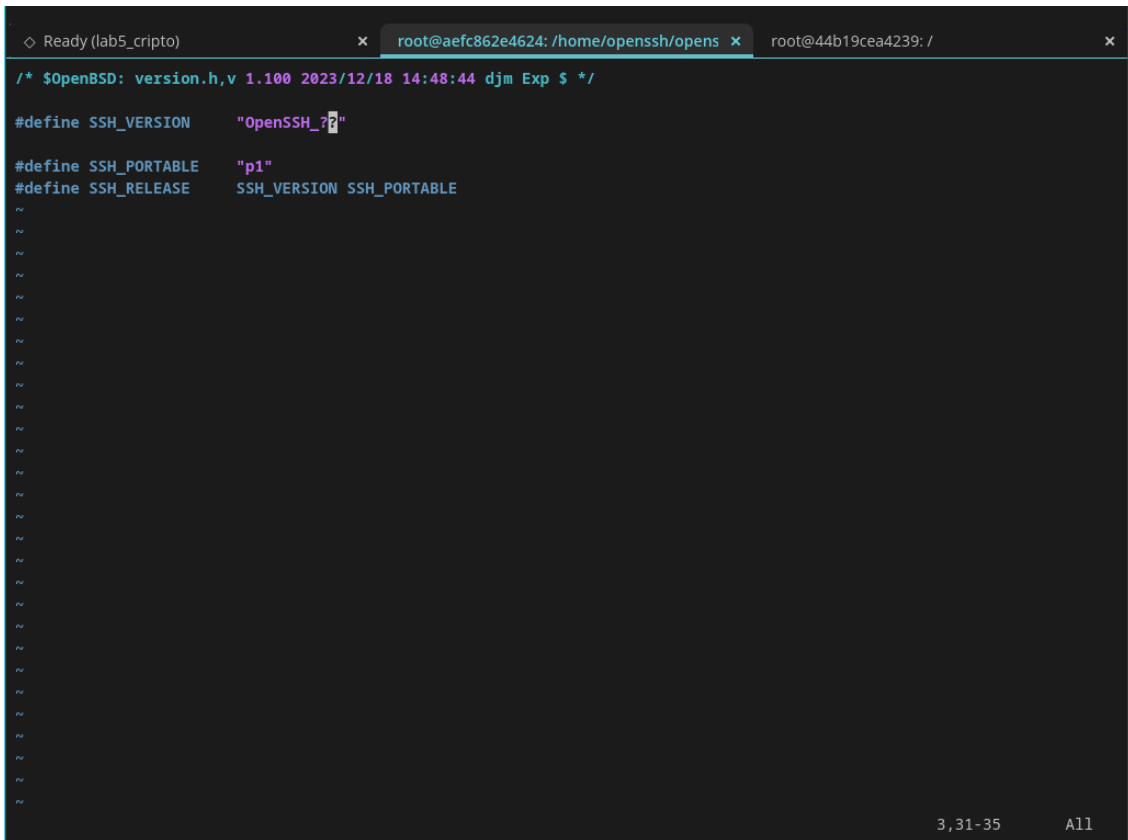


Figura 14: Modificación de SSH_VERSION a .openSSH?.en version.h

Paso 5: Configuración del entorno de compilación. Se ejecutó el script `./configure` con los parámetros `--with-pam`, `--with-zlib` y `--with-ssl-dir=/usr` para preparar el entorno de compilación según las librerías disponibles en el contenedor.



Figura 15: Ejecución de `./configure` para preparar la compilación

Paso 6: Compilación del código fuente. Se compiló el proyecto con `make -j$(nproc)` para aprovechar todos los núcleos disponibles y reducir el tiempo de compilación.

```
root@aefc862e4624:/home/openssh/openssh-9.6p1# make -j$(nproc)
```

Figura 16: Compilación con make -j\$(nproc)

Paso 7: Reemplazo del binario y verificación de la versión. Se copió el binario compilado sobre /usr/bin/ssh y se verificó con `ssh -V` que la versión reportada sea `OpenSSH_??p1`. Luego se realizó la conexión al servidor S1 (`ssh prueba@172.21.0.10`).

```
root@aefc862e4624:/home/openssh/openssh-9.6p1# cp ssh /usr/bin/ssh
root@aefc862e4624:/home/openssh/openssh-9.6p1# ssh -V
OpenSSH_??p1, OpenSSL 1.1.1f 31 Mar 2020
root@aefc862e4624:/home/openssh/openssh-9.6p1# ssh prueba@172.21.0.10
prueba@172.21.0.10's password:
```

Figura 17: Reemplazo del binario, verificación de versión y conexión al servidor

Paso 8: Captura del tráfico en Wireshark. En la captura se puede observar que el cliente (172.21.0.13) envía al servidor (172.21.0.10) el paquete de protocolo con el banner `SSH-2.0-OpenSSH_??`, confirmando que el tráfico fue replicado exitosamente con la versión ?.

tcp.port == 22 && ssh					
No.	Time	Source	Destination	Protocol	Length Info
4	0.000420945	172.21.0.13	172.21.0.10	SSHv2	86 Client: Protocol (SSH-2.0-OpenSSH_??)
6	0.001673734	172.21.0.10	172.21.0.13	SSHv2	107 Server: Protocol (SSH-2.0-OpenSSH_9.0p1)

Figura 18: Captura Wireshark mostrando el banner SSH-2.0-OpenSSH_?? en el tráfico replicado

3. Desarrollo (Parte 3)

3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)

El paquete *Key Exchange Init* (KEXINIT) es grande porque contiene listas de algoritmos separadas por comas para cada categoría: intercambio de llaves, llaves de host, cifrado, MAC y compresión. El tamaño del paquete es directamente proporcional a la cantidad de texto en esas listas. Para reducirlo a menos de 300 bytes se modificó el archivo `myproposal.h` en el código fuente de OpenSSH dentro del servidor S1, que es donde se definen esas listas por defecto.

En la imagen se puede observar el archivo antes de la modificación. Las macros `KEX_SERVER_KEX` y `KEX_DEFAULT_PK_ALG` contienen largas listas con 9 y 14 algoritmos respectivamente, lo que genera strings de cientos de bytes solo en esas dos categorías.

```

*/

#define KEX_SERVER_KEX \
    "sntrup761x25519-sha512@openssh.com," \
    "curve25519-sha256," \
    "curve25519-sha256@libssh.org," \
    "ecdh-sha2-nistp256," \
    "ecdh-sha2-nistp384," \
    "ecdh-sha2-nistp521," \
    "diffie-hellman-group-exchange-sha256," \
    "diffie-hellman-group16-sha512," \
    "diffie-hellman-group18-sha512," \
    "diffie-hellman-group14-sha256"

#define KEX_CLIENT_KEX KEX_SERVER_KEX

#define KEX_DEFAULT_PK_ALG \
    "ssh-ed25519-cert-v01@openssh.com," \
    "ecdsa-sha2-nistp256-cert-v01@openssh.com," \
    "ecdsa-sha2-nistp384-cert-v01@openssh.com," \
    "ecdsa-sha2-nistp521-cert-v01@openssh.com," \
    "sk-ssh-ed25519-cert-v01@openssh.com," \
    "sk-ecdsa-sha2-nistp256-cert-v01@openssh.com," \
    "rsa-sha2-512-cert-v01@openssh.com," \
    "rsa-sha2-256-cert-v01@openssh.com," \
    "ssh-ed25519," \
    "ecdsa-sha2-nistp256," \
    "ecdsa-sha2-nistp384," \
    "ecdsa-sha2-nistp521," \
    "sk-ssh-ed25519@openssh.com," \
    "sk-ecdsa-sha2-nistp256@openssh.com," \
    "rsa-sha2-512," \
    "rsa-sha2-256"

#define KEX_SERVER_ENCRYPT \
    "chacha20-poly1305@openssh.com," \
    "aes128-ctr,aes192-ctr,aes256-ctr," \
    "aes128-gcm@openssh.com,aes256-gcm@openssh.com"

#define KEX_CLIENT_ENCRYPT KEX_SERVER_ENCRYPT

```

Figura 19: Archivo myproposal.h original con las listas completas de algoritmos

Se reemplazaron todas las listas por versiones mínimas de un solo algoritmo por categoría, tal como se ve en la imagen siguiente:

- KEX_SERVER_KEX: se dejó solo `diffie-hellman-group14-sha256` (28 bytes)
- KEX_DEFAULT_PK_ALG: se dejó solo `ssh-ed25519-cert-v01@openssh.com` (34 bytes)
- KEX_SERVER_ENCRYPT: se redujo a `aes128-ctr,aes192-ctr,aes256-ctr` (31 bytes)
- KEX_SERVER_MAC: se dejó solo `hmac-sha2-256` (13 bytes)

```
#define KEX_SERVER_KEX \
    "diffie-hellman-group14-sha256"

#define KEX_CLIENT_KEX KEX_SERVER_KEX

#define KEX_DEFAULT_PK_ALG \
    "ssh-ed25519-cert-v01@openssh.com,"

#define KEX_SERVER_ENCRYPT \
    "aes128-ctr,aes192-ctr,aes256-ctr,"

#define KEX_CLIENT_ENCRYPT KEX_SERVER_ENCRYPT

#define KEX_SERVER_MAC \
    "hmac-sha2-256,"

#define KEX_CLIENT_MAC KEX_SERVER_MAC
```

Figura 20: Archivo `myproposal.h` modificado con listas mínimas de algoritmos

El tamaño del KEXINIT depende directamente del largo de los strings de algoritmos. Con la configuración original, solo `KEX_SERVER_KEX` ocupa más de 200 bytes. Al dejar un algoritmo por categoría, la suma total cae a unos 106 bytes, resultando en un paquete de aproximadamente 168 bytes. Luego se recompiló con `make sshd` y se reemplazó el binario en `S1`.

4. Desarrollo (Parte 4)

4.1. Explicación OpenSSH en general

OpenSSH es una implementación de código abierto del protocolo SSH (Secure Shell), diseñada para permitir comunicación remota cifrada entre dos equipos. Opera sobre TCP, generalmente en el puerto 22. La conexión comienza con un handshake donde ambos extremos

intercambian versiones, negocian algoritmos y generan una llave de sesión compartida usando Diffie-Hellman o variantes de curvas elípticas. Una vez establecido el canal cifrado, se realiza la autenticación del usuario (por contraseña o llave pública) y luego se habilita la sesión.

4.2. Capas de Seguridad en OpenSSH

OpenSSH implementa seguridad en tres niveles distintos del protocolo:

- **Confidencialidad:** todo el tráfico se cifra con algoritmos simétricos como `aes128-ctr` o `chacha20-poly1305`. Nadie que intercepte el tráfico puede leer el contenido una vez que el canal está establecido.
- **Integridad:** cada paquete incluye un código de autenticación de mensaje (MAC) calculado con algoritmos como `hmac-sha2-256`. Si el paquete fue modificado en tránsito, el receptor lo detecta y descarta la conexión.
- **Autenticidad:** el servidor presenta su llave de host durante el handshake. Si el cliente ya conoce esa llave (guardada en `known_hosts`), puede verificar que está hablando con el servidor correcto y no con un intermediario. La autenticación del usuario puede hacerse por contraseña o por par de llaves pública/privada.

La disponibilidad y el no repudio no son garantizados directamente por el protocolo. SSH no incluye mecanismos propios contra denegación de servicio, y al autenticarse por contraseña no existe evidencia criptográfica irrefutable de que fue ese usuario quien inició sesión, a diferencia de la autenticación por llave pública donde sí existe esa garantía.

4.3. Identificación de que protocolos no se cumplen

A partir del tráfico analizado en este laboratorio, la disponibilidad no se puede garantizar mediante el protocolo SSH por sí solo, ya que depende de factores externos como la configuración del servidor y la red. Tampoco se cumple completamente el no repudio cuando se usa autenticación por contraseña, ya que no existe evidencia criptográfica que vincule la sesión a una identidad específica. En cambio, con autenticación por llave pública el no repudio sí se puede argumentar, ya que solo el poseedor de la llave privada pudo haber firmado el intercambio.

Conclusiones y comentarios

En este laboratorio se analizó el comportamiento del protocolo SSH a nivel de tráfico de red utilizando diferentes versiones de OpenSSH en contenedores Docker. Se comprobó que cada versión genera una huella digital (HASSH) distinta a partir de los algoritmos que anuncia en el paquete KEXINIT, lo que permite identificar el cliente con precisión. Además, se demostró que modificando el código fuente de OpenSSH es posible alterar tanto la cadena

de versión mostrada en el banner como el conjunto de algoritmos anunciados, logrando reducir el tamaño del KEXINIT del servidor a menos de 300 bytes. Esto evidencia que la información visible antes del cifrado puede ser manipulada libremente, lo que tiene implicaciones directas en técnicas de fingerprinting y en la capacidad de un atacante o analista de inferir información sobre el sistema.